Jenkins & Playwright — Beginner's Guide

> **Audience**: developers and technical people who are new to Jenkins and Playwright. You should be comfortable with Docker basics (images, containers, `docker-compose up`) but no prior Jenkins or E2E testing experience is assumed.
>
> **What this guide covers**: what Playwright and Jenkins are, how they're set up in HelpDesk Pro, where every environment variable lives, and step-by-step what happens when you trigger a test run — both locally and on OpenShift.
>
> **For Helm template details**: this guide focuses on *concepts and flows*. For the line-by-line YAML template walkthroughs, see the OpenShift & Helm Deep-Dive Guide.

Table of Contents

- Jenkins & Playwright — Beginner's Guide
  - Table of Contents
  - Part 1 — What Are These Tools?
    - 1.1 What is Playwright?
    - 1.2 What is Jenkins?
    - 1.3 How They Work Together
    - 1.4 Key Terms Glossary
  - Part 2 — Playwright in This Project
    - 2.1 The e2e/ Directory
    - 2.2 Test Configuration
    - 2.3 The Page Object Pattern
    - 2.4 Test Data and Cleanup
    - 2.5 How a Test Run Works (Step by Step)
  - Part 3 — Jenkins in This Project
    - 3.1 The Custom Jenkins Docker Image
    - 3.2 Jenkins Plugins
    - 3.3 Configuration as Code (JCasC)
      - Section 1: OIDC Authentication (Login via Keycloak)
      - Section 2: Role-Based Access Control
      - Section 3: Stored Credentials
      - Section 4: Job Definition (Job DSL)
    - 3.4 The "Run-Playwright-Tests" Job
      - Step 1: Parameters (What you can configure before clicking "Build")
      - Step 2: SCM — Clone the Code
      - Step 3: Credentials Binding — Inject Secrets Securely
      - Step 4: Build Step — Run the Tests
      - Step 5: Post-Build — Publish Results
  - Part 4 — Where Do the Environment Variables Live?
    - 4.1 The Variable Story
      - Scene 1 — Image Build Time
      - Scene 2 — Container Startup
      - Scene 3 — The Two-URL Keycloak Problem
      - Scene 4 — JCasC Configures Jenkins
      - Scene 5 — You Trigger a Build
      - Scene 6 — The Tests Run
      - The Full Chain at a Glance
    - 4.2 Complete Variable Map
      - Jenkins Container Variables
      - Playwright Test Variables (inside the build step)
      - Firefox-Specific Variables (set by `ci-entrypoint.sh`)
  - Part 5 — What Happens When You Run It
    - 5.1 Locally (docker-compose)

        - Starting the Stack
        - Jenkins Startup Sequence
        - Why Port 9090?
    - 5.2 On OpenShift
        - Deployment
        - Jenkins Startup on OpenShift
        - Key Differences from Local
- Part 6 — Demo Walkthrough Script
    - Prerequisites
    - The Demo (Step by Step)
        - Step 1 — Show the Application
        - Step 2 — Open Jenkins
        - Step 3 — Explore the Dashboard
        - Step 4 — Trigger a Build
        - Step 5 — Watch the Build
        - Step 6 — View the Report
        - Step 7 — (Optional) Show the Shortcut: Running from Command Line
- Part 7 — How It All Connects
    - Project Files Quick Reference
- Part 8 — Going Deeper
    - In This Project
    - Official Documentation

Part 1 — What Are These Tools?

1.1 What is Playwright?

Playwright is a tool that tests your application the way a real person would use it.

When you write "unit tests", you're testing small pieces of code in isolation — a single function, a single component. That's great for catching bugs in logic, but it can't tell you if the login page *actually works* when a user types their password and clicks "Sign In".

That's where **end-to-end (E2E) testing** comes in. E2E tests launch a real web browser, navigate to your application, click buttons, fill in forms, and verify that the right things appear on screen. If a user can do it, an E2E test can automate it.

Playwright is Microsoft's open-source E2E testing framework. It:

- **Controls real browsers** — Chromium (Chrome), Firefox, and WebKit (Safari)
- **Runs "headless"** — the browser runs without a visible window, which is perfect for automated testing on a server (like Jenkins). You can also run it "headed" (visible window) for debugging.
- **Handles modern web apps** — it waits for elements to appear, handles single-page app navigation, and deals with authentication flows automatically
- **Generates reports** — HTML reports with screenshots, traces, and detailed failure information

> **In this project**: Playwright tests live in the `e2e/` folder. They test the HelpDesk Pro application by logging in as different users (employee, admin), submitting tickets, checking the admin dashboard, and verifying that everything works end to end.

1.2 What is Jenkins?

Jenkins is a server that runs your tests (and other tasks) automatically.

You *could* run Playwright tests manually from your terminal every time you want to check if the app works. But that's tedious. Jenkins is a **CI/CD server** (Continuous Integration / Continuous Delivery) that:

- **Provides a web dashboard** — you click a button to trigger a test run, and watch the results in your browser
- **Runs jobs automatically** — you can schedule tests, or trigger them when code is pushed to Git
- **Keeps history** — every test run is recorded with logs, reports, and artifacts
- **Manages credentials securely** — passwords and secrets are stored encrypted, not in plain text
- **Controls access** — different users can have different permissions (admin, tester, read-only)

Think of Jenkins as a **robot assistant** that sits on a server, waiting for you to say "run the tests". When you do, it:

1. Downloads the latest code from Git
2. Runs the Playwright tests
3. Collects the results
4. Shows you a nice HTML report

> **In this project**: Jenkins is deployed alongside HelpDesk Pro (both locally and on OpenShift). It comes pre-configured with a "Run-Playwright-Tests" job — no manual setup needed.

1.3 How They Work Together

Here's the relationship in one picture:

```
┌──────────────────────────────────────────────────────────┐
│                        JENKINS                           │
│                   (the orchestrator)                     │
│                                                          │
│  1. You click "Build" in the Jenkins web UI              │
│  2. Jenkins clones the Git repository                    │
│  3. Jenkins runs the Playwright test scripts             │
│                          │                               │
│                          ▼                               │
│   ┌──────────────────────────────────────────────┐       │
│   │              PLAYWRIGHT                      │       │
│   │          (the test runner)                   │       │
│   │                                              │       │
│   │  4. Launches a headless Chromium browser     │       │
│   │  5. Navigates to HelpDesk Pro                │       │
│   │  6. Logs in via Keycloak                     │       │
│   │  7. Submits tickets, checks dashboard, etc.  │       │
│   │  8. Reports pass/fail for each test          │       │
│   └──────────────────────────────────────────────┘       │
│                          │                               │
│  9. Jenkins collects the HTML report                     │
│ 10. You view results in the Jenkins UI                   │
└──────────────────────────────────────────────────────────┘
```

Jenkins is the orchestrator — it triggers, manages, and reports. **Playwright is the test runner** — it does the actual browser automation. Jenkins doesn't know *how* to test a web app. Playwright doesn't know *when* to run or where to put the results. Together, they form a complete automated testing pipeline.

1.4 Key Terms Glossary

You'll encounter these terms throughout this guide and in the project's configuration files. Each one is explained in detail later, but here's a quick reference:

Term	What It Means
E2E (End-to-End)	Testing the full application flow as a user would experience it — from the browser through the backend to the database and back.
Headless Browser	A browser running without a visible window. Used for automated testing on servers where there's no screen.
CI/CD	**Continuous Integration** (automatically test code changes) / **Continuous Delivery** (automatically deploy tested code). Jenkins handles the CI part here.
SCM (Source Code Management)	A system that tracks code changes — in practice, this means **Git**. When Jenkins says "SCM", it means "pull the code from the Git repository".

Term	What It Means
DSL (Domain-Specific Language)	A mini-language designed for one purpose. Jenkins uses **Job DSL** — a Groovy-based language for defining jobs in code instead of clicking through the UI.
JCasC (Jenkins Configuration as Code)	A Jenkins plugin that lets you define *all* Jenkins settings (security, plugins, jobs, credentials) in a YAML file. Instead of configuring Jenkins through its web UI, you write a YAML file and Jenkins configures itself on startup.
OIDC (OpenID Connect)	A login protocol. Instead of Jenkins managing its own usernames and passwords, it delegates authentication to Keycloak. When you click "Log in" on Jenkins, you're redirected to Keycloak's login page.
PVC (PersistentVolumeClaim)	Kubernetes storage that survives Pod restarts. Jenkins uses a PVC to keep its job history and build data even if the Jenkins container is restarted or moved.
Page Object Model	A test design pattern where each page of the app gets its own class with methods like `login()`, `submitTicket()`, etc. Keeps tests clean and reusable.
Job (Jenkins)	A configured task that Jenkins can run. Our project has one job: "Run-Playwright-Tests".
Build (Jenkins)	A single execution of a job. Each time you click "Build", Jenkins creates a new build with a number (#1, #2, #3...).

Part 2 — Playwright in This Project

2.1 The e2e/ Directory

All Playwright tests and configuration live in the `e2e/` folder at the project root. Here's what each part does:

```
e2e/
├── playwright.config.ts       ← Main configuration: which browsers, which tests, what settings
├── global-setup.ts            ← Runs BEFORE all tests (cleans up old test data)
├── global-teardown.ts         ← Runs AFTER all tests (cleans up again)
├── cleanup.ts                 ← Shared cleanup logic (deletes test tickets from the database)
├── package.json               ← npm dependencies (Playwright + TypeScript)
│
├── fixtures/
│   └── test-data.ts           ← Test data: ticket titles, user credentials, expected values
│
├── pages/                     ← Page Objects (one class per app page)
│   ├── login.page.ts          ← Keycloak login page interactions
│   ├── submit-ticket.page.ts  ← Employee ticket submission form
│   ├── admin-dashboard.page.ts← Admin dashboard table, filters, stats
│   └── navigation.page.ts    ← Shared navigation (sidebar, menus)
│
├── tests/                     ← Actual test files, organized by user role
│   ├── auth.setup.ts          ← Authentication setup (logs in and saves session)
│   ├── employee/              ← Tests that run as an employee user
│   │   ├── submit-ticket-form.spec.ts
│   │   ├── submit-ticket-validation.spec.ts
│   │   └── my-tickets.spec.ts
│   ├── admin/                 ← Tests that run as an admin user
│   │   ├── tickets-table.spec.ts
│   │   ├── dashboard-stats.spec.ts
│   │   ├── filter-sort.spec.ts
│   │   ├── assign-ticket.spec.ts
│   │   └── update-status.spec.ts
│   ├── auth/                  ← Login/logout tests
│   │   ├── login.employee.spec.ts
│   │   ├── login.admin.spec.ts
│   │   ├── logout.employee.spec.ts
│   │   └── logout.admin.spec.ts
```

```
│   └── navigation/            ← Navigation tests
│
└── scripts/                   ← Shell scripts for CI automation
    ├── ci-entrypoint.sh       ← Main entry point for Jenkins (orchestrates everything)
    ├── install-deps.sh        ← Installs npm packages + Playwright browsers
    └── run-tests.sh           ← Runs Playwright with the right flags
```

2.2 Test Configuration

The file `e2e/playwright.config.ts` is where Playwright learns *what* to test and *how*. The most important concept here is **projects**.

A **project** is a combination of **browser** + **user role**. For example:

Project Name	Browser	User Role	What It Tests
`setup-chromium`	Chrome	—	Logs in as employee + admin, saves sessions
`setup-firefox`	Firefox	—	Same, but in Firefox
`employee-chromium`	Chrome	employee1	Employee tests (submit tickets, view my tickets)
`employee-firefox`	Firefox	employee1	Same employee tests, in Firefox
`admin-chromium`	Chrome	admin1	Admin tests (dashboard, assign, filter, stats)
`admin-firefox`	Firefox	admin1	Same admin tests, in Firefox

The **setup projects** run first (they log in and save the browser session to a file). Then the **test projects** reuse that saved session — so each test doesn't need to log in again.

Key settings from the config:

```
fullyParallel: false,        // Tests run one at a time (not in parallel)
retries: process.env.CI ? 1 : 0,  // On CI (Jenkins), retry failed tests once
workers: process.env.CI ? 1 : undefined, // On CI, use 1 worker (saves memory)
reporter: process.env.CI ? 'html' : 'list', // On CI, generate HTML report
```

The `baseURL` (which app URL to test) comes from the `BASE_URL` environment variable, defaulting to `http://localhost:4200`.

> **How `page` and Page Objects actually work**
>
> Every test receives a `page` object — a programmatic handle to a **live, headless browser tab**. This is the single object through which all browser interactions happen.
>
> When a test calls `page.goto(BASE_URL)`, Playwright genuinely opens a browser, sends an HTTP request to that URL, and loads whatever HTML/JS the server returns — exactly the same as a human visiting the URL. Angular is just a web server from Playwright's perspective.
>
> When a test calls `page.locator('#some-id')`, Playwright asks the live browser: **"find the element with this CSS selector in whatever HTML you currently have loaded."** The selector is evaluated against the real DOM at the moment the call executes — not against a snapshot or a mirror. Nothing is pre-loaded or cached.
>
> **Page Objects contain no HTML or DOM.** A class like `LoginPage` is just a named collection of selector strings (`'#username'`, `'#password'`) and wrapper methods (`login()`). When you write `this.usernameInput = page.locator('#username')`, you're storing an address — a promise to ask the live browser for that element when you actually need it.
>
> The practical consequence: if a developer changes a button's ID in the HTML, the selector string in the Page Object no longer matches anything in the live browser, and the test fails. That's intentional — the test is telling you the UI changed.
>
> If you only need to interact with an element once, you don't need a Page Object at all. You can call `page.locator('#some-id')` directly in the test. Page Objects are purely a code-organisation pattern for selectors you'll reuse across many tests.

2.3 The Page Object Pattern

Instead of writing raw browser commands in every test, the project uses the **Page Object Model**. Each page of the app has a corresponding class in `e2e/pages/` that wraps the browser interactions.

For example, `e2e/pages/login.page.ts`:

```
export class LoginPage {
  constructor(page: Page) {
    this.usernameInput = page.locator('#username');
    this.passwordInput = page.locator('#password');
    this.submitButton = page.locator('#kc-login');
  }

  async login(username: string, password: string): Promise<void> {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}
```

> **How Playwright connects to the app**: Playwright opens a real browser and navigates to `BASE_URL` (e.g. `http://localhost:4200`). The Angular app, seeing no active session, immediately **redirects the browser to Keycloak's login page**. `LoginPage` targets the HTML elements on *that Keycloak page* — the `#username`, `#password`, and `#kc-login` IDs belong to Keycloak's own login form, not to Angular. So `login.page.ts` is really a "Keycloak login page" object. The `BASE_URL` environment variable is the single wire connecting Playwright to whichever environment is under test.

Then in a test, instead of repeating those three steps every time, you just write:

```
const loginPage = new LoginPage(page);
await loginPage.login('employee1', 'password123');
```

Why this matters: if the login page HTML changes (say, the button ID changes from `#kc-login` to `#login-btn`), you only fix it in one place (the Page Object), not in every test.

2.4 Test Data and Cleanup

Test data is defined in `e2e/fixtures/test-data.ts`. All test tickets have titles starting with `E2E` — this prefix is important because the cleanup logic uses it.

Cleanup (`e2e/cleanup.ts`) runs before *and* after the test suite. It:

1. Gets an admin token from Keycloak
2. Fetches all tickets from the API
3. Deletes any ticket whose title starts with `E2E`

This ensures tests start with a clean slate and don't leave junk data behind.

2.5 How a Test Run Works (Step by Step)

When Playwright tests execute (whether from your terminal or Jenkins), here's the exact sequence:

```
1. GLOBAL SETUP (global-setup.ts)
   └─ Calls cleanup.ts → deletes any leftover "E2E *" tickets from previous runs

2. AUTH SETUP (tests/auth.setup.ts)
   ├─ Launches a browser
   ├─ Navigates to the app → redirected to Keycloak login
   ├─ Logs in as employee1 → saves browser session to .auth/employee-chromium.json
```

```
    └─ Logs in as admin1 → saves browser session to .auth/admin-chromium.json

3. TEST EXECUTION (runs each project in order)
   ├─ employee-chromium: loads saved employee session → runs employee/*.spec.ts
   ├─ employee-firefox: loads saved employee session → runs employee/*.spec.ts (Firefox)
   ├─ admin-chromium: loads saved admin session → runs admin/*.spec.ts
   └─ admin-firefox: loads saved admin session → runs admin/*.spec.ts (Firefox)

4. GLOBAL TEARDOWN (global-teardown.ts)
   └─ Calls cleanup.ts → deletes all "E2E *" tickets created during tests

5. REPORTING
   ├─ HTML report generated in e2e/playwright-report/
   ├─ Screenshots saved for failed tests in e2e/test-results/
   └─ Traces saved for retried tests (detailed step-by-step recording)
```

Part 3 — Jenkins in This Project

3.1 The Custom Jenkins Docker Image

Jenkins doesn't come with Node.js or Playwright browsers out of the box. Since our tests need both, the project builds a **custom Jenkins image** (defined in `jenkins/Dockerfile`) that includes everything:

```
Official Jenkins LTS (Java 17)
    │
    ├── + Node.js 20          ← Needed to run npm and Playwright
    ├── + Chromium browser     ← Pre-installed so tests don't download it every time
    ├── + Firefox browser      ← Same — pre-installed for speed
    ├── + Jenkins plugins      ← From jenkins/plugins.txt (see section 3.2)
    └── + Writable directories ← For OpenShift compatibility (runs as random non-root user)
```

Why pre-install browsers? Playwright browsers are ~400MB. If Jenkins had to download them every time a test runs, each build would waste minutes just downloading. By baking them into the Docker image, tests start immediately.

Key lines in the Dockerfile explained:

```
# Pre-install browsers at image build time
ENV PLAYWRIGHT_BROWSERS_PATH=/ms-playwright
COPY e2e/package.json e2e/package-lock.json* /tmp/e2e/
RUN cd /tmp/e2e && npm ci && npx playwright install --with-deps chromium firefox
```

This copies the `e2e/package.json` into the image, installs dependencies, and downloads Chromium + Firefox to `/ms-playwright`. The `--with-deps` flag also installs system libraries the browsers need (fonts, graphics libraries, etc.).

```
# Disable the setup wizard — JCasC handles everything
ENV JAVA_OPTS="... -Djenkins.install.runSetupWizard=false ..."
```

Normally, when you start Jenkins for the first time, it shows a setup wizard asking you to create an admin user and pick plugins. We skip this because JCasC configures everything automatically.

```
# Tell Jenkins where to find the JCasC config file
ENV CASC_JENKINS_CONFIG="/var/jenkins_home/casc_configs/"
```

This points Jenkins to the directory where the Configuration as Code YAML file will be mounted.

3.2 Jenkins Plugins

The file `jenkins/plugins.txt` lists the plugins installed in the custom image:

Plugin	What It Does
`oic-auth`	**OpenID Connect authentication** — lets Jenkins delegate login to Keycloak instead of managing its own passwords
`role-strategy`	**Role-based access control** — lets us map Keycloak roles (helpdesk-admin, helpdesk-tester) to Jenkins permissions
`configuration-as-code`	**JCasC** — configure Jenkins entirely from a YAML file (no manual UI setup)
`git`	**Git integration** — lets Jenkins clone repositories (this is the SCM plugin)
`workflow-aggregator`	**Pipeline support** — the core pipeline engine (even though we use a freestyle job, this is a common dependency)
`htmlpublisher`	**HTML reports** — publishes the Playwright HTML report so you can view it in Jenkins
`nodejs`	**Node.js tool** — registers Node.js as a build tool in Jenkins
`job-dsl`	**Job DSL** — lets us define jobs in Groovy code (inside the JCasC YAML) instead of creating them manually through the UI
`credentials`	**Credentials store** — securely stores passwords, tokens, secrets
`credentials-binding`	**Credentials binding** — injects stored credentials into build steps as environment variables

3.3 Configuration as Code (JCasC)

This is the most important piece to understand. Instead of clicking through Jenkins' web UI to set up security, credentials, and jobs, everything is declared in a single YAML file: `jenkins/casc/jenkins.yaml`.

When Jenkins starts, it reads this file and configures itself. **No manual setup, no clicking, completely reproducible.**

The JCasC file has four main sections:

Section 1: OIDC Authentication (Login via Keycloak)

```
securityRealm:
  oic:
    clientId: "helpdesk-jenkins"
    clientSecret: "${JENKINS_OIDC_CLIENT_SECRET}"
    # ... Keycloak endpoint URLs ...
```

What this does: When someone visits Jenkins and isn't logged in, Jenkins redirects them to Keycloak's login page. After the user logs in with their Keycloak credentials, Keycloak sends Jenkins a token that says "this is user admin1, they have roles helpdesk-admin and employee". Jenkins uses this to determine who they are and what they can do.

The `helpdesk-jenkins` client: This is a Keycloak client specifically for Jenkins (defined in the Keycloak realm). It's different from `helpdesk-frontend` (for the Angular app) and `helpdesk-backend` (for the .NET API). Each application that authenticates with Keycloak gets its own client.

The client secret (`${JENKINS_OIDC_CLIENT_SECRET}`): This is a shared password between Jenkins and Keycloak that proves Jenkins is who it claims to be. The `${...}` syntax means "read this from an environment variable" — so the secret isn't hardcoded in the YAML file.

Section 2: Role-Based Access Control

```
authorizationStrategy:
  roleBased:
    roles:
      global:
        - name: "admin"
          permissions: ["Overall/Administer"]
          entries:
            - group: "helpdesk-admin"
        - name: "tester"
          permissions: ["Job/Build", "Job/Read", ...]
          entries:
            - group: "helpdesk-tester"
        - name: "readonly"
          permissions: ["Overall/Read"]
          entries:
            - group: "authenticated"
```

What this does: Maps Keycloak roles to Jenkins permissions.

If your Keycloak role is...	You get Jenkins role...	Which means you can...
`helpdesk-admin`	admin	Do everything (full admin access)
`helpdesk-tester`	tester	Trigger builds, view results, cancel jobs
(any authenticated user)	readonly	View the dashboard (read-only)

So `admin1` (who has the `helpdesk-admin` role in Keycloak) gets full Jenkins admin access. `tester1` (who has `helpdesk-tester`) can trigger test runs but can't change Jenkins configuration. `employee1` (who has neither) gets read-only access.

Section 3: Stored Credentials

```
credentials:
  system:
    domainCredentials:
      - credentials:
          - string:
              id: "employee-username"
              secret: "${EMPLOYEE_USERNAME:-employee1}"
          - string:
              id: "employee-password"
              secret: "${EMPLOYEE_PASSWORD:-password123}"
          # ... same for admin ...
```

What this does: Stores the test user credentials (employee1/password123, admin1/password123) in Jenkins' encrypted credential store. When the Playwright tests run, they need to log into the HelpDesk app — these credentials tell them which username and password to use.

The `:-` syntax: `${EMPLOYEE_USERNAME:-employee1}` means "use the `EMPLOYEE_USERNAME` environment variable, but if it's not set, default to `employee1`". This makes the config work both locally (where the defaults are fine) and on OpenShift (where you might need different credentials).

Section 4: Job Definition (Job DSL)

```
jobs:
  - script: |
      job('Run-Playwright-Tests') {
        description('Runs Playwright E2E tests against HelpDesk Pro')
        parameters {
```

```
            stringParam('BASE_URL', ...)
            choiceParam('BROWSER_PROJECT', ['chromium', 'all', 'firefox', ...])
            stringParam('TEST_RETRIES', '1', ...)
          }
          scm { git { ... } }
          wrappers { credentialsBinding { ... } }
          steps { shell('cd e2e && ./scripts/ci-entrypoint.sh') }
          publishers { publishHtml { ... } }
        }
```

What this does: Automatically creates a Jenkins job called "Run-Playwright-Tests" when Jenkins starts. This is written in **Job DSL** — a Groovy-based language for defining jobs in code. The next section breaks this down in detail.

3.4 The "Run-Playwright-Tests" Job

This is the job you'll interact with. It's auto-created by JCasC (Section 4 above). Here's what each part does:

Step 1: Parameters (What you can configure before clicking "Build")

When you click "Build with Parameters", you see three options:

Parameter	Default	What It Controls
BASE_URL	`http://frontend:8080` (local) or the OpenShift frontend route URL	The URL Playwright will test against. Change this if you want to test a different environment.
BROWSER_PROJECT	`chromium`	Which browser(s) to test. Options: `chromium` (fast, recommended), `firefox`, `all` (both — needs more memory).
TEST_RETRIES	`1`	How many times to retry a failed test before marking it failed. `1` means each test gets two chances.

Step 2: SCM — Clone the Code

```
scm {
  git {
    remote { url('https://github.com/maguiarr/Helpdesk.git') }
    branch('main')
  }
}
```

SCM = Source Code Management, which in practice means **Git**. This tells Jenkins: "Before running the tests, clone the `main` branch of the repository." This ensures Jenkins always tests the latest code, including any changes to the test files themselves.

Why clone? Jenkins needs the `e2e/` folder (test scripts), but it doesn't have the project code by default — it just has the test runner (Playwright) pre-installed in its Docker image. The SCM step fetches everything.

Step 3: Credentials Binding — Inject Secrets Securely

```
wrappers {
  credentialsBinding {
    string('EMPLOYEE_USERNAME', 'employee-username')
    string('EMPLOYEE_PASSWORD', 'employee-password')
    string('ADMIN_USERNAME', 'admin-username')
    string('ADMIN_PASSWORD', 'admin-password')
  }
}
```

This takes the credentials stored in Section 3 (JCasC) and makes them available as **environment variables** during the build. So when the test script runs, it can read `$EMPLOYEE_USERNAME` and `$EMPLOYEE_PASSWORD` to log into the app.

Why not just hardcode them? Credentials binding means passwords are:

- Encrypted at rest in Jenkins
- Masked in build logs (you'll see `****` instead of the actual password)
- Managed centrally (change in one place, applies to all jobs)

Step 4: Build Step — Run the Tests

```
steps {
  shell('''
    cd e2e
    chmod +x scripts/*.sh
    ./scripts/ci-entrypoint.sh
  ''')
}
```

This is the actual execution. It:

1. Changes to the `e2e/` directory (inside the cloned repo)
2. Makes the shell scripts executable
3. Runs `ci-entrypoint.sh`, which orchestrates everything (see the flow below)

What `ci-entrypoint.sh` does (the CI pipeline within the build step):

```
ci-entrypoint.sh
│
├── Step 1: install-deps.sh
│   ├── npm ci (install Node.js packages)
│   └── Check if Playwright browsers exist at /ms-playwright (they do — pre-installed in image)
│
├── Step 2: Health check
│   └── Ping the backend's /healthz endpoint up to 30 times
│       (wait for the app to be ready before running tests)
│
├── Step 3: Export credentials
│   └── Set EMPLOYEE_USERNAME, EMPLOYEE_PASSWORD, etc. from Jenkins env vars
│
└── Step 4: run-tests.sh
    ├── Build the Playwright command based on BROWSER_PROJECT
    ├── Set retries, reporter, workers from env vars
    └── Execute: npx playwright test --project=employee-chromium --project=admin-chromium ...
```

Step 5: Post-Build — Publish Results

```
publishers {
  publishHtml {
    report('e2e/playwright-report') {
      reportName('Playwright Report')
      alwaysLinkToLastBuild(true)
    }
  }
  archiveArtifacts {
    pattern('e2e/test-results/**')
    allowEmpty(true)
```

```
    }
  }
```

After the tests finish (pass or fail), Jenkins:

- **Publishes the HTML report** — the Playwright HTML report (with test details, screenshots, traces) becomes viewable directly in the Jenkins UI as "Playwright Report"
- **Archives artifacts** — test result files (screenshots, trace files) are saved as downloadable build artifacts

---

Part 4 — Where Do the Environment Variables Live?

The variables can feel overwhelming because they're defined in **five different places** and pass through several handoffs before a Playwright test ever runs. This section tells the full story — *why* the system is designed this way, and *what* each handoff does. The quick-reference tables are in section 4.2 once the story makes sense.

4.1 The Variable Story

Follow Jenkins from first boot to first test run. At each stage, a new group of variables enters the picture.

Scene 1 — Image Build Time

When it happens: `docker build` (locally) or the GitHub Actions image build push (CI/CD). **File in charge**: `jenkins/Dockerfile`

Before any container starts, the Docker image is assembled. The Dockerfile bakes in three variables that are the same in *every* environment — they're structural facts about the image itself:

```
┌──────────────────────────────────────────────────────────────────┐
│                       JENKINS IMAGE (built once)                 │
│                                                                  │
│  PLAYWRIGHT_BROWSERS_PATH = /ms-playwright                       │
│    → tells every subsequent process where Chromium/Firefox live  │
│                                                                  │
│  CASC_JENKINS_CONFIG = /var/jenkins_home/casc_configs/           │
│    → tells Jenkins where to look for its YAML configuration      │
│                                                                  │
│  JAVA_OPTS = "-Xmx384m ... -DrunSetupWizard=false ..."           │
│    → JVM memory limits + skips the first-time setup wizard       │
└──────────────────────────────────────────────────────────────────┘
```

> **Key idea**: These variables never change between environments. They describe how the image works internally, not how it connects to the outside world. You can override them at runtime, but you rarely need to.

Scene 2 — Container Startup

When it happens: `make up` (locally) or `helm install` (OpenShift). **Files in charge**: `docker-compose.yml` / `helm/templates/jenkins-deployment.yaml`

When the container starts, the runtime environment injects the **"address book"** — where is everything else? These are the variables that differ between local and OpenShift, because the network addresses are different.

```
LOCAL (docker-compose.yml injects):            OPENSHIFT (jenkins-deployment.yaml injects):
──────────────────────────────────             ────────────────────────────────────────────
KEYCLOAK_URL          = http://keycloak:8180   KEYCLOAK_URL      = https://{keycloak-route}
KEYCLOAK_BROWSER_URL  = http://localhost:8180  (no KEYCLOAK_BROWSER_URL — see Scene 3)
JENKINS_OIDC_CLIENT_SECRET = jenkins-secret    JENKINS_OIDC_CLIENT_SECRET = from K8s Secret
DEFAULT_BASE_URL      = http://frontend:8080   DEFAULT_BASE_URL  = https://{frontend-route}
```

```
JENKINS_ADMIN_PASSWORD = admin                JENKINS_ADMIN_PASSWORD = from K8s Secret
                                              HEALTH_CHECK_URL   = http://backend:8080/healthz
```

Notice that locally, secrets are plain text in `docker-compose.yml` (convenient for development). On OpenShift, they come from Kubernetes Secrets (encrypted, managed by the platform).

> **Why is `HEALTH_CHECK_URL` only set on OpenShift?** Locally, `ci-entrypoint.sh` falls back to `$BASE_URL/api/healthz`, which works fine. On OpenShift, `DEFAULT_BASE_URL` is a public HTTPS route, but the healthcheck needs to reach the backend *internally* without going through the public internet — so it uses the internal Kubernetes service name instead.

Scene 3 — The Two-URL Keycloak Problem

> This is the trickiest part of the whole variable setup. Read it carefully — it trips up most people.

When you log into Jenkins, **two completely different network paths** are involved:

1. **Your browser** opens the Jenkins page, Jenkins redirects it to the Keycloak login page. Your browser needs a URL *your laptop* can reach.
2. **Jenkins itself** (running inside Docker) needs to call Keycloak's API to exchange the login token and verify your identity. Jenkins needs a URL *the Docker container* can reach.

Locally, these are completely different addresses:

```
YOUR LAPTOP                        DOCKER NETWORK
___________                        ______________

                                   ┌──────────────────────┐
Your browser ── localhost:8180 ──►│                      │
                                   │   Keycloak container  │
Jenkins JVM  ── keycloak:8180 ──►│   (internal hostname) │
                                   └──────────────────────┘
```

Your browser has no idea what `keycloak` means as a hostname — that name only exists inside the Docker virtual network. Similarly, `localhost:8180` inside the Jenkins container points to Jenkins itself, not Keycloak.

Solution: two separate variables.

Variable	Used For	Local Value	OpenShift Value
`KEYCLOAK_BROWSER_URL`	Browser redirects (login, logout, issuer discovery)	`http://localhost:8180`	*(not set)*
`KEYCLOAK_URL`	Server-to-server API calls (token exchange, JWKS, userinfo)	`http://keycloak:8180`	`https://{keycloak-route}`

On OpenShift, both Jenkins and your browser use the public HTTPS Route URL — it's accessible from everywhere, so one variable handles both.

```
LOCAL:
  Browser redirects  → KEYCLOAK_BROWSER_URL (http://localhost:8180)
  Token exchange     → KEYCLOAK_URL         (http://keycloak:8180)

OPENSHIFT:
  Browser redirects  → KEYCLOAK_URL         (https://keycloak-route-host)
  Token exchange     → KEYCLOAK_URL         (https://keycloak-route-host)
  (one URL works for both — KEYCLOAK_BROWSER_URL is not set)
```

Scene 4 — JCasC Configures Jenkins

When it happens: ~30 seconds after the container starts, while Jenkins initializes. **File in charge**: `jenkins/casc/jenkins.yaml`

Jenkins reads the YAML file mounted at `CASC_JENKINS_CONFIG` and configures itself. Every `${VAR_NAME}` in the YAML is substituted with the environment variable from Scene 2. Three important things happen here:

① Keycloak login is wired up

The JCasC YAML uses the two Keycloak URL variables precisely:

```
KEYCLOAK_BROWSER_URL → authorizationServerUrl, endSessionUrl, issuer  (browser-facing)
KEYCLOAK_URL         → tokenServerUrl, jwksServerUrl, userInfoServerUrl (server-to-server)
JENKINS_OIDC_CLIENT_SECRET → clientSecret (the shared password with Keycloak)
```

After this, clicking "Log in" on Jenkins redirects to Keycloak, and the token handshake works.

② Test credentials are stored in the Jenkins vault

JCasC reads `EMPLOYEE_USERNAME`, `EMPLOYEE_PASSWORD`, `ADMIN_USERNAME`, `ADMIN_PASSWORD` from the environment (using defaults of `employee1` / `password123` if not set) and stores them **encrypted** inside Jenkins' credential store. From this point on, the plain-text values are gone — Jenkins only exposes them to build steps through a masking mechanism.

```
                    ┌──────────────────────────────┐
Env var             │     JENKINS CREDENTIAL       │
EMPLOYEE_USERNAME   │  id: "employee-username"     │  ← encrypted at rest
EMPLOYEE_PASSWORD   │  id: "employee-password"     │  ← masked as **** in logs
ADMIN_USERNAME      │  id: "admin-username"        │
ADMIN_PASSWORD      │  id: "admin-password"        │
                    └──────────────────────────────┘
```

③ The test job is created

Job DSL (inside the same JCasC YAML) creates the "Run-Playwright-Tests" job. `DEFAULT_BASE_URL` is read here and becomes the **default value** for the job's `BASE_URL` parameter — so when a user opens "Build with Parameters", the right URL is already pre-filled.

Scene 5 — You Trigger a Build

When it happens: You click "Build with Parameters" in the Jenkins UI. **File in charge**: The job definition (inside `jenkins/casc/jenkins.yaml`)

This is the only stage where a *human* injects variables. You can override three things:

```
┌──────────────────────────────────────────────────────────────────┐
│                  "Build with Parameters" dialog                  │
│                                                                  │
│  BASE_URL        [http://frontend:8080          ]  ← pre-filled from │
│                                                        DEFAULT_BASE_URL│
│  BROWSER_PROJECT [chromium ▼]                                    │
│  TEST_RETRIES    [1                             ]                 │
└──────────────────────────────────────────────────────────────────┘
```

Want to test a staging environment instead of the local one? Change `BASE_URL`. Want to run only Firefox? Change `BROWSER_PROJECT`. Otherwise, leave the defaults. These parameters only affect the test run — they don't change Jenkins configuration.

Scene 6 — The Tests Run

When it happens: Jenkins executes the shell build step. **File in charge**: `e2e/scripts/ci-entrypoint.sh`

The credentials from Scene 4's vault are now injected into the build environment by the credentials binding wrapper, masked as `****` in the logs:

```
Vault "employee-username" → env var EMPLOYEE_USERNAME  (shown as **** in logs)
Vault "employee-password" → env var EMPLOYEE_PASSWORD  (shown as **** in logs)
Vault "admin-username"    → env var ADMIN_USERNAME     (shown as **** in logs)
Vault "admin-password"    → env var ADMIN_PASSWORD     (shown as **** in logs)
```

Then `ci-entrypoint.sh` takes over and adds the remaining runtime vars:

```
ci-entrypoint.sh also sets:
  TEST_REPORTER   = "list,html"               ← output formats
  TEST_WORKERS    = 1                         ← serialize tests (memory constraint)
  NODE_OPTIONS    = "--max-old-space-size=256" ← cap Node.js heap
  TEST_RETRIES    = from job parameter (Scene 5)
  BROWSER_PROJECT = from job parameter (Scene 5)

  If Firefox: (OpenShift security workarounds)
    MOZ_DISABLE_CONTENT_SANDBOX = 1
    MOZ_DISABLE_GMP_SANDBOX = 1
    MOZ_DISABLE_SOCKET_PROCESS_SANDBOX = 1
    MOZ_DISABLE_RDD_SANDBOX = 1
    HOME = /tmp/firefox-home     ← writable by any UID
    FONTCONFIG_CACHE = /tmp/firefox-home/.cache/fontconfig
```

At the end, `npx playwright test` runs. Playwright reads `BASE_URL`, `EMPLOYEE_USERNAME`, `EMPLOYEE_PASSWORD`, `ADMIN_USERNAME`, `ADMIN_PASSWORD`, `PLAYWRIGHT_BROWSERS_PATH` — all of which have been passed down through the chain above.

The Full Chain at a Glance

```
Stage 1: docker build       → bakes PLAYWRIGHT_BROWSERS_PATH, CASC_JENKINS_CONFIG, JAVA_OPTS
              |
Stage 2: container starts   → injects KEYCLOAK_URL, KEYCLOAK_BROWSER_URL,
              |                       JENKINS_OIDC_CLIENT_SECRET, DEFAULT_BASE_URL, ...
              |
Stage 3: JCasC reads YAML   → wires Keycloak login (KEYCLOAK_URL + KEYCLOAK_BROWSER_URL)
              |                stores test creds encrypted in vault (EMPLOYEE_*, ADMIN_*)
              |                creates job with DEFAULT_BASE_URL as default parameter
              |
Stage 4: user clicks Build  → injects BASE_URL, BROWSER_PROJECT, TEST_RETRIES
              |
Stage 5: build step runs    → vault creds injected as masked env vars
                              ci-entrypoint.sh adds TEST_WORKERS, TEST_REPORTER,
                              NODE_OPTIONS, Firefox sandbox workarounds
                              └─ Playwright runs with the complete set
```

4.2 Complete Variable Map

Here's every environment variable, what it does, and exactly where it's defined:

Jenkins Container Variables

Variable	Purpose	Local (`docker-compose.yml`)	OpenShift (`jenkins-deployment.yaml`)

Variable	Purpose	Local (`docker-compose.yml`)	OpenShift (`jenkins-deployment.yaml`)
`JAVA_OPTS`	JVM memory limits, disable setup wizard, CSP for HTML reports	Set in `Dockerfile`	Set in Deployment `env:`
`CASC_JENKINS_CONFIG`	Path to JCasC config files	Set in `Dockerfile` (`/var/jenkins_home/casc_configs/`)	Set in Deployment `env:`
`KEYCLOAK_URL`	Keycloak URL for **server-to-server** calls (token exchange, JWKS)	`http://keycloak:8180` (internal Docker network)	`https://{keycloak-route-host}` (public URL)
`KEYCLOAK_BROWSER_URL`	Keycloak URL for **browser redirects** (login page)	`http://localhost:8180` (user's browser)	*(not set — same as `KEYCLOAK_URL` since both are public on OpenShift)*
`JENKINS_OIDC_CLIENT_SECRET`	Shared secret between Jenkins and Keycloak	`jenkins-secret` (plain text in compose)	From `jenkins-secret.yaml` (Kubernetes Secret)
`DEFAULT_BASE_URL`	Default app URL for Playwright tests	`http://frontend:8080` (internal)	`https://{frontend-route-host}` (public)
`HEALTH_CHECK_URL`	Backend health endpoint for pre-test check	*(not set — script defaults to `$BASE_URL/api/healthz`)*	`http://helpdesk-pro-backend:8080/healthz` (internal)
`JENKINS_ADMIN_PASSWORD`	Fallback admin password if Keycloak is unavailable	`admin`	From `jenkins-secret.yaml`
`PLAYWRIGHT_BROWSERS_PATH`	Where Playwright browsers are installed	Set in `Dockerfile` (`/ms-playwright`)	Same (inherited from image)

Local-only variables (needed because Docker networking works differently):

Variable	Purpose	Why It's Needed Locally
`KEYCLOAK_BROWSER_URL`	The Keycloak URL as seen by your browser	Locally, your browser can't reach `keycloak:8180` (that's the Docker internal hostname). It needs `localhost:8180`. On OpenShift, the Route's public URL works for both.

Playwright Test Variables (inside the build step)

Variable	Purpose	Set By
`BASE_URL`	The app URL Playwright navigates to	Job parameter (user sets it, or defaults to `DEFAULT_BASE_URL`)
`EMPLOYEE_USERNAME`	Test user login name	JCasC credentials → credentials binding
`EMPLOYEE_PASSWORD`	Test user login password	JCasC credentials → credentials binding
`ADMIN_USERNAME`	Admin test user login name	JCasC credentials → credentials binding
`ADMIN_PASSWORD`	Admin test user login password	JCasC credentials → credentials binding
`BROWSER_PROJECT`	Which browser projects to run	Job parameter
`TEST_RETRIES`	Number of retry attempts	Job parameter
`TEST_WORKERS`	Number of parallel test workers	Set by `ci-entrypoint.sh` (= `1` to save memory)
`TEST_REPORTER`	Output format	Set by `ci-entrypoint.sh` (= `list,html`)

Variable	Purpose	Set By
`CI`	Signals "running in CI mode"	*(set automatically by Jenkins)*
`NODE_OPTIONS`	Node.js memory limit	Set by `ci-entrypoint.sh` (`--max-old-space-size=256`)

Firefox-Specific Variables (set by `ci-entrypoint.sh`)

If Firefox tests run, the CI script sets these to work around OpenShift security restrictions:

Variable	Purpose
`MOZ_DISABLE_CONTENT_SANDBOX`	Disables Firefox content sandbox (OpenShift blocks `clone(CLONE_NEWUSER)`)
`MOZ_DISABLE_GMP_SANDBOX`	Disables GMP plugin sandbox
`MOZ_DISABLE_SOCKET_PROCESS_SANDBOX`	Disables socket process sandbox
`MOZ_DISABLE_RDD_SANDBOX`	Disables RDD sandbox
`HOME`	Set to `/tmp/firefox-home` (writable directory for Firefox profile)
`FONTCONFIG_CACHE`	Writable font cache for headless Firefox

> These Firefox variables are only relevant when running on OpenShift, where containers run as a random non-root user ID with restricted security contexts.

Part 5 — What Happens When You Run It

5.1 Locally (docker-compose)

Here's exactly what happens when you run the system locally:

Starting the Stack

```
make up    # or: docker-compose up -d --build
```

This starts **five services** in dependency order:

```
1. postgres        (port 5432) — Database starts first
      ↓ healthy
2. keycloak        (port 8180) — Identity provider, imports realm config
      ↓ healthy                ↓ healthy
3. backend         (port 8080) 5. jenkins    (port 9090)
      ↓ healthy                   CI server, connects to
4. frontend        (port 4200)    keycloak for login
      Angular app (nginx)
```

> **Note**: Backend → Frontend and Jenkins are **parallel branches** — both depend on Keycloak, but Jenkins does **not** depend on Frontend or Backend. In practice, all five services start roughly together; Docker just waits for each service's healthcheck dependencies before marking it healthy.

Jenkins Startup Sequence

Jenkins is the slowest to start (~2 minutes). Here's what happens inside:

```
1. Jenkins JVM starts
   └─ JAVA_OPTS: -Xmx384m, setup wizard disabled, CSP relaxed for HTML reports
```

```
2. Plugins load
   └─ oic-auth, configuration-as-code, job-dsl, htmlpublisher, git, etc.

3. JCasC applies (reads /var/jenkins_home/casc_configs/jenkins.yaml)
   ├─ Configures Keycloak OIDC login
   │   └─ Uses KEYCLOAK_URL (http://keycloak:8180) for token endpoints
   │   └─ Uses KEYCLOAK_BROWSER_URL (http://localhost:8180) for browser redirects
   ├─ Sets up role mapping (admin → helpdesk-admin, tester → helpdesk-tester)
   ├─ Stores test credentials (employee1, admin1)
   └─ Creates "Run-Playwright-Tests" job via Job DSL
       └─ DEFAULT_BASE_URL (http://frontend:8080) becomes the job's default BASE_URL

4. Jenkins is ready
   └─ Web UI available at http://localhost:9090
```

Why Port 9090?

In `docker-compose.yml`, Jenkins maps `9090:8080` — the container's port 8080 (Jenkins' internal port) is exposed as 9090 on your machine. This avoids conflicting with the backend, which also uses port 8080.

5.2 On OpenShift

On OpenShift, the flow is similar but the deployment mechanism is different.

Deployment

```
helm install helpdesk-pro ./helm -f helm/values.openshift.yaml
```

Helm renders all templates using values from `values.yaml` + `values.openshift.yaml`, creating ~22 Kubernetes resources including the Jenkins Deployment, Service, Route, PVC, Secret, and ConfigMap.

Jenkins Startup on OpenShift

```
1. Deployment creates a Pod
   ├─ Mounts PVC (5Gi) at /var/jenkins_home (persistent storage)
   ├─ Mounts ConfigMap (JCasC) at /var/jenkins_home/casc_configs/
   └─ Mounts emptyDir at /dev/shm (64Mi shared memory for Chromium)

2. Environment variables injected from Deployment spec:
   ├─ KEYCLOAK_URL = https://{keycloak-route-host}   (public HTTPS URL)
   ├─ DEFAULT_BASE_URL = https://{frontend-route-host} (public HTTPS URL)
   ├─ HEALTH_CHECK_URL = http://helpdesk-pro-backend:8080/healthz (internal)
   ├─ JENKINS_OIDC_CLIENT_SECRET = from jenkins-secret (Kubernetes Secret)
   └─ JENKINS_ADMIN_PASSWORD = from jenkins-secret

3. JCasC applies (same process as local, but URLs are HTTPS public routes)
   └─ All Keycloak endpoints point to public route (no separate BROWSER_URL needed)

4. Health probes start checking
   ├─ Readiness: GET /login (passes after ~90s)
   └─ Liveness: GET /login (passes after ~120s)

5. Route becomes active
   └─ Jenkins accessible at: https://helpdesk-pro-jenkins-{namespace}.apps.{cluster}/
```

Key Differences from Local

Aspect	Local (docker-compose)	OpenShift
URLs	`http://localhost:PORT`	`https://{auto-generated-route-host}`
Keycloak URLs	Two variables: `KEYCLOAK_URL` (internal) + `KEYCLOAK_BROWSER_URL` (browser)	One variable: `KEYCLOAK_URL` (public HTTPS works for both)
Secrets	Plain text in `docker-compose.yml`	Kubernetes Secrets (encrypted, reference by name)
Storage	Docker volume (`jenkins_home:`)	PVC (Kubernetes-managed persistent storage)
JCasC source	File mount: `./jenkins/casc/jenkins.yaml` mounted into container	ConfigMap: `jenkins-casc-configmap.yaml` template rendered by Helm
Network	Docker bridge network (services reach each other by name)	Kubernetes Services + DNS (same idea, different implementation)
Startup time	~2 minutes	~2 minutes (same, but longer probe delays before OpenShift considers it "ready")

> **For the full Helm template walkthroughs** — what each YAML field does, how the Deployment, Service, Route, PVC, Secret, and ConfigMap templates work — see the OpenShift & Helm Deep-Dive Guide, Section 3.5 (Jenkins).

Part 6 — Demo Walkthrough Script

Use this as a script when demonstrating Jenkins + Playwright to others. Each step includes what to show and what to say.

Prerequisites

- All services running: `make up` (wait ~3 minutes for everything to be healthy)
- Verify services are up: `make logs` (check for errors)

Service URLs:

Service	URL
HelpDesk App	http://localhost:4200
Jenkins	http://localhost:9090
Keycloak Admin	http://localhost:8180

The Demo (Step by Step)

Step 1 — Show the Application

1. Open http://localhost:4200 in your browser
2. You'll be redirected to the Keycloak login page
3. Log in as `employee1` / `password123`
4. You'll land on the ticket submission page

> **Say**: "*This is HelpDesk Pro — a support ticket application. Employees can submit tickets, and admins can manage them. Right now I'm logged in as an employee. Let me submit a ticket so we have some data.*"

5. Submit a ticket with any title and description
6. Point out the "My Tickets" section below

> **Say**: "*Now the question is: how do we know this application actually works correctly? We could click around manually every time we make a change, but that's slow and error-prone. That's where automated testing comes in.*"

Step 2 — Open Jenkins

1. Open http://localhost:9090 in a new tab

2. You'll be redirected to Keycloak's login page (same login page as the app!)

> **Say**: "*Notice we're redirected to Keycloak — the same identity provider the app uses. Jenkins doesn't manage its own user accounts. It delegates authentication to Keycloak via OpenID Connect. This means our Keycloak users and roles work in Jenkins too.*"

3. Log in as `admin1` / `password123`
4. You'll land on the Jenkins dashboard

> **Say**: "*I logged in as admin1, who has the helpdesk-admin role in Keycloak. Jenkins maps that to full admin access. If I had logged in as tester1, I'd only be able to trigger builds, not change settings.*"

Step 3 — Explore the Dashboard

1. Point out the "Run-Playwright-Tests" job in the job list

> **Say**: "*This job was created automatically when Jenkins started — we didn't click anything to set it up. It's defined in code, in a Configuration-as-Code YAML file. If we delete this Jenkins instance and start a new one, the job reappears automatically.*"

Step 4 — Trigger a Build

1. Click on "Run-Playwright-Tests"
2. Click "Build with Parameters" in the left sidebar
3. Show the three parameters:
   - **BASE_URL**: `http://frontend:8080` — "*This is the URL Playwright will test. It's the internal Docker network address of our frontend.*"
   - **BROWSER_PROJECT**: `chromium` — "*We can test with Chromium, Firefox, or both. Chromium is faster and uses less memory.*"
   - **TEST_RETRIES**: `1` — "*If a test fails, Playwright will retry it once. This helps with flaky tests.*"
4. Click **Build**

Step 5 — Watch the Build

1. Click on the build number (e.g., `#1`) in the Build History
2. Click **Console Output** in the left sidebar

> **Say**: "*Let's watch what happens in real time.*"

3. Point out key moments in the log as they appear:
   - **"Installing npm dependencies"** — "*First it installs the Node.js packages.*"
   - **"Browsers already installed at /ms-playwright"** — "*The browsers are pre-installed in the Docker image, so we skip the download.*"
   - **"Health check against http://frontend:8080/api/healthz"** — "*Before running tests, it checks that the application backend is healthy. No point testing if the app is down.*"
   - **"Backend is healthy!"** — "*We're good to go.*"
   - **"Running Playwright tests"** — "*Now Playwright launches a headless Chromium browser and starts testing.*"
   - **Lines like** `✓ [employee-chromium] › employee/submit-ticket-form.spec.ts` — "*Each test file runs and reports pass/fail.*"
   - **"X passed"** at the end — "*All tests passed!*"

Step 6 — View the Report

1. Go back to the build page (click "Back to Run-Playwright-Tests" or the build number)
2. Click **Playwright Report** in the left sidebar

> **Say**: "*Jenkins publishes the Playwright HTML report. You can drill into each test, see what steps it took, and if any failed, see screenshots of what the browser showed at the moment of failure.*"

3. Browse the report — click on a test to show its details

Step 7 — (Optional) Show the Shortcut: Running from Command Line

> **Say**: "*You can also run these exact same tests from your terminal, without Jenkins:*"

```
cd e2e
npm install
npx playwright test --project=employee-chromium --project=admin-chromium
npx playwright show-report    # Opens the HTML report in your browser
```

> **Say**: "*Jenkins adds value by providing a web dashboard, history, credential management, and team access. But the tests themselves are just Playwright — they can run anywhere Node.js is installed.*"

Part 7 — How It All Connects

Here's the complete picture of how all the pieces fit together:

```
┌──────────────────────────────────────────┐
│              YOUR BROWSER                 │
│                                           │
│   1. Open Jenkins (localhost:9090)        │
│   2. Redirect to Keycloak login           │
│   3. Log in as admin1                     │
│   4. Click "Build with Parameters"        │
│   5. Click "Build"                        │
└──────────────────┬────────────────────────┘
                   │
                   ▼
┌──────────────────────────────────────────┐
│              JENKINS                      │
│                                           │
│   6.  Clone Git repo (SCM step)           │
│   7.  Inject credentials (binding)        │
│   8.  Run ci-entrypoint.sh                │
│       ├── npm install                     │
│       ├── Health check backend            │
│       └── npx playwright test             │
│               │                           │
└               │                           ┘
                │
                ▼
┌──────────────────────────────────────────┐
│            PLAYWRIGHT                     │
│                                           │
│   9.  Launch headless Chromium            │
│   10. Navigate to HelpDesk app            │
│   11. Keycloak login (automated)          │
│   12. Run test scenarios:                 │
│       ├── Submit ticket ✓                 │
│       ├── Check dashboard ✓               │
│       ├── Assign ticket ✓                 │
│       └── Filter/sort ✓                   │
│   13. Generate HTML report + screenshots  │
└──────────────────┬────────────────────────┘
                   │
                   ▼
┌──────────────────────────────────────────┐
│            BACK IN JENKINS                │
│                                           │
│   14. Collect Playwright report           │
│   15. Publish as "Playwright Report"      │
│   16. Archive test artifacts              │
│   17. Mark build SUCCESS or FAILURE       │
│                                           │
```

```
                        │   → You view the results in Jenkins UI   │
                        └──────────────────────────────────────────┘
```

Project Files Quick Reference

File	Purpose
`jenkins/Dockerfile`	Custom Jenkins image with Node.js + Playwright browsers
`jenkins/plugins.txt`	Jenkins plugins to install
`jenkins/casc/jenkins.yaml`	JCasC: OIDC login, roles, credentials, job definition (local)
`jenkins/jobs/playwright-tests.groovy`	Reference-only copy of the Job DSL script (not used at runtime)
`e2e/playwright.config.ts`	Playwright config: browsers, projects, test settings
`e2e/tests/auth.setup.ts`	Logs in as employee + admin, saves session for other tests
`e2e/cleanup.ts`	Deletes test tickets before/after the test run
`e2e/pages/*.page.ts`	Page Objects: login, submit-ticket, admin-dashboard, navigation
`e2e/fixtures/test-data.ts`	Test data: ticket templates, user credentials, expected values
`e2e/scripts/ci-entrypoint.sh`	CI orchestrator: install → health check → export creds → run tests
`e2e/scripts/run-tests.sh`	Builds and runs the `npx playwright test` command
`docker-compose.yml`	Local dev environment (all 5 services including Jenkins)
`helm/templates/jenkins-*.yaml`	OpenShift deployment templates (6 files)
`helm/values.yaml`	Helm values: Jenkins image, resources, Keycloak client, persistence

---

Part 8 — Going Deeper

In This Project

- **OpenShift & Helm Deep-Dive Guide** — Full Helm template walkthroughs for all components. The Jenkins section (3.5) covers every YAML field in detail.
- **Deployment Troubleshooting** — Resolved deployment issues, root causes, and diagnostic commands.

Official Documentation

- **Playwright**: https://playwright.dev/ — The official docs have excellent guides on writing tests, debugging, and configuration.
- **Jenkins Configuration as Code (JCasC)**: https://www.jenkins.io/projects/jcasc/ — How JCasC works, schema reference, and examples.
- **Jenkins Job DSL**: https://plugins.jenkins.io/job-dsl/ — The DSL plugin documentation and API reference for defining jobs in code.
- **OpenID Connect (OIDC) Basics**: https://openid.net/developers/how-connect-works/ — Understanding the protocol Jenkins uses to authenticate with Keycloak.